

VSPICE Server 서비스 중지 지연 개선

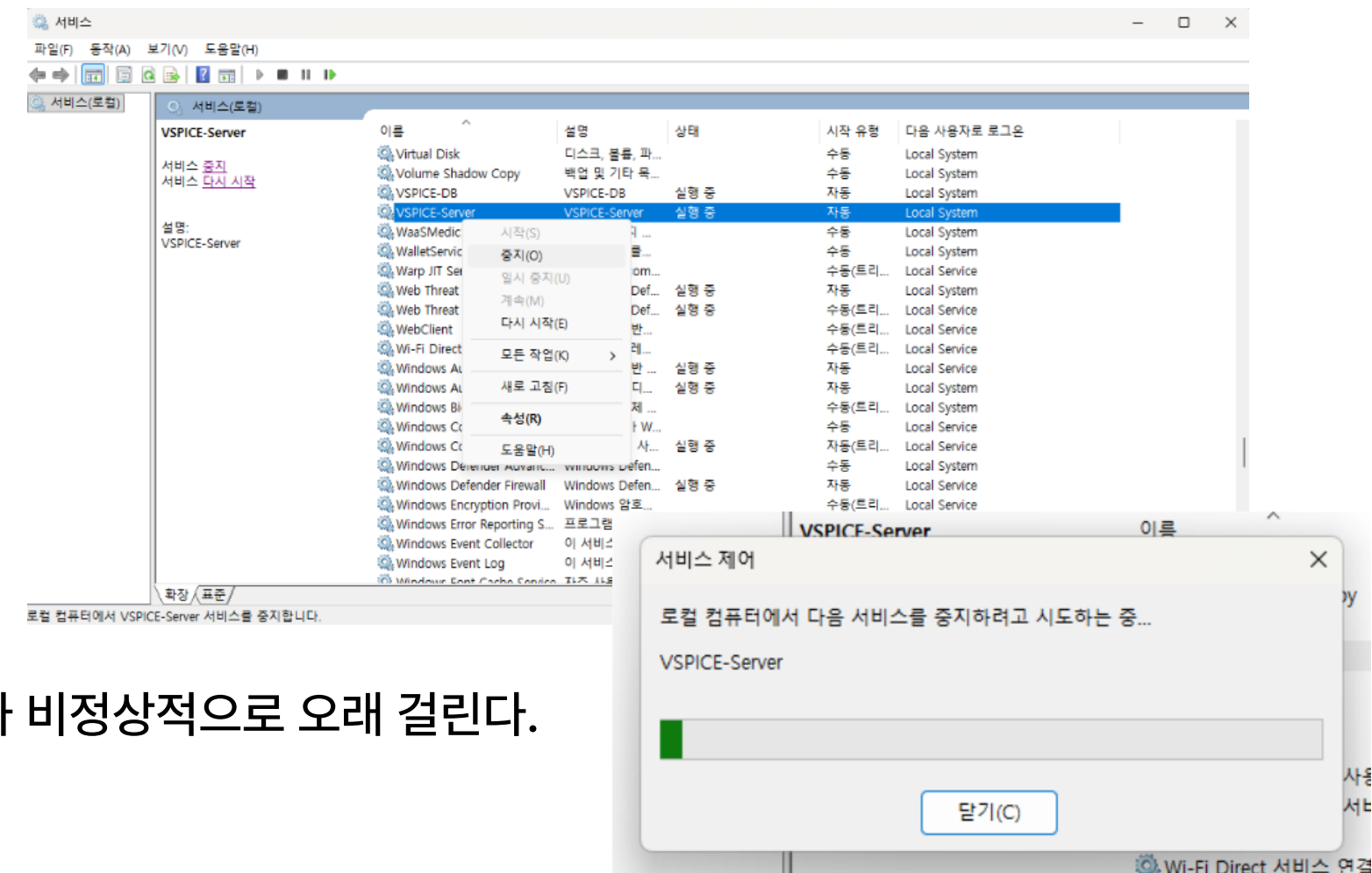
문제 현상 정리

서비스 재부팅의 필요성

- 고객사 패치 전달 시 WAR 파일 교체가 필요하다.
- 교체된 WAR 파일을 반영하려면 서비스 재부팅 필요.
- 재부팅 후 새 버전으로 서비스가 기동된다.

증상

- 작업 관리자 / 서비스 관리자에서 VSPICE Server 서비스 중지가 비정상적으로 오래 걸린다.
- 과거에는 빨리 종료됐는데 어느 시점부터 느려졌다.
- 고객사 저사양 PC에서는 종료 도중 "멈춤" 상태가 발생
- 서비스가 깔끔히 내려가지 못하고, 결국 PC를 재부팅해야만 복구되는 경우가 존재한다.



문제 현상 정리

로그 분석

VSPICE-Server 종료 로그 확인 결과, Catalina 종료 시퀀스 자체는 정상적으로 수행됨.
종료 요청 후 Catalina는 pause → stop → destroy 단계까지 약 4.26초 만에 완료

다만 종료 과정에서 웹 애플리케이션이 생성한 백그라운드 스레드 3개가 정상 종료되지 않았다는 경고가 발생 !

즉, 문제의 핵심은 Catalina 종료가 느린 것이 아니라,
서비스 종료 후에도 일부 장수 스레드가 남아 JVM 종료를 지연시키는 구조였다.

이로 인해 전체 서비스 종료 시간이 약 231초(약 3분 51초)까지 증가



```
12-Jun-2026 16:24:25.854 WARNING [Thread-36] org.apache.catalina.loader.WebappClassLoaderBase.clearReferencesThreads 웹 애플리케이션 [vspice]이(가) [Thread-34](이)라는 이름의 스레드를 시작시킨 것으로 보이지만, 해당 스레드를 중지시키지 못했습니다. 이는 메모리 누수를 유발할 가능성이 큼니다. 해당 스레드의 스택 트레이스:  
java.base/java.lang.Thread.sleep0(Native Method)  
java.base/java.lang.Thread.sleep(Thread.java:509)  
com.suresoft.qscroll.acst.util.AcstProjectGarbageFileThreadProcessor.run(AcstProjectGarbageFileThreadProcessor.java:100)
```

```
12-Jun-2026 16:24:21.681 WARNING [Thread-36] org.apache.catalina.loader.WebappClassLoaderBase.clearReferencesThreads 웹 애플리케이션 [RtmsLinkServer]이(가) [Thread-2](이)라는 이름의 스레드를 시작시킨 것으로 보이지만, 해당 스레드를 중지시키지 못했습니다. 이는 메모리 누수를 유발할 가능성이 큼니다. 해당 스레드의 스택 트레이스:  
java.base/java.lang.Thread.sleep0(Native Method)  
java.base/java.lang.Thread.sleep(Thread.java:509)  
com.suresoft.qscroll.acst.link.server.processor.ThreadWorkerProcessor.run(ThreadWorkerProcessor.java:111)
```

```
com.suresoft.qscroll.acst.link.server.processor.ThreadWorkerProcessor.run(ThreadWorkerProcessor.java:111)  
12-Jun-2026 16:24:25.853 WARNING [Thread-36] org.apache.catalina.loader.WebappClassLoaderBase.clearReferencesThreads 웹 애플리케이션 [vspice]이(가) [Thread-33](이)라는 이름의 스레드를 시작시킨 것으로 보이지만, 해당 스레드를 중지시키지 못했습니다. 이는 메모리 누수를 유발할 가능성이 큼니다. 해당 스레드의 스택 트레이스:  
java.base/java.lang.Thread.sleep0(Native Method)  
java.base/java.lang.Thread.sleep(Thread.java:509)  
com.suresoft.qscroll.acst.util.Session.thread.SessionManagement.run(SessionManagement.java:75)
```

02 원인 분석

원인 분석

문제 지점 분석

문제의 세 스레드 모두 동일한 안티패턴을 공유한다

extends Thread / while(true) 무한루프 / 종료 플래그/인터럽트 처리 없음 / 비데몬으로 start()

즉, 외부에서 정상적으로 멈출 방법 자체가 설계에 없었던걸로 확인된다.

```
public class AcstProjectGarbageFileThreadProcessor extends Thread { 5개 사용 위치

/**
 * 프로젝트 생성시 만들어지는 3개 폴더영역을 검사하여 삭제 대기처리된 폴더를 추출하여 삭제 작업을
 */
@Override
public void run() {
    while (true) {
        try {

            // Get Garbage Check Folder
            File projectRoot = new File( /*pathname*/ AcstUtil.getAcstInternalPathOnRoot

            //F:
            //F:
            File

            try {
                Thread.sleep( /*millis*/ 500);
            } catch (InterruptedException e) {
                log.error("**", e);
            }

        } catch (Exception e) {
            log.error("**", e);
        }
    }
}
```

AcstProjectGarbageFileThreadProcessor

프로젝트 삭제 시 남는 가비지 폴더/파일을
주기적으로(0.5초) 청소하는 싱글톤 스레드

```
public class SessionManagement extends Thread {

public void run() {
    while(true) {
        try {
            if(SessionList != null) {
                synchronized (SessionList) {
                    for(HttpSession session : SessionList.values()) {
                        sessionService.compareSession(session, SessionActionLocation.BackEnd);
                    }
                }
            }

            sleep( /*millis*/ 3000);
        } catch (NullPointerException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

SessionManagement

3초마다 전체 세션을 백엔드와
비교 동기화하는 싱글톤 스레드

```
ThreadWorkerProcessor.java x AcstApplicationStartupAfterProcess.java

49
50 @Override
51 public void run() {
52     while (true) {
53         try {
54             synchronized (jobQueue) {
55                 if (jobQueue.size() > 0) {
56                     ProjectInfo projectInfo = pro
57                 }
58             }
59             Thread.sleep(100);
60         } catch (InterruptedException e) {
61             log.error("ThreadWorkerProcessor::InterruptedException - ",e);
62         } catch (Exception e) {
63             log.error("Exception in ThreadWorkerProcessor::run() - ", e);
64             // 예외가 발생해도 루프를 계속 실행
65         }
66     }
}
```

ThreadWorkerProcessor

업로드 잡 큐를 0.1초마다 폴링하여
처리하는 싱글톤 스레드

02 원인 분석

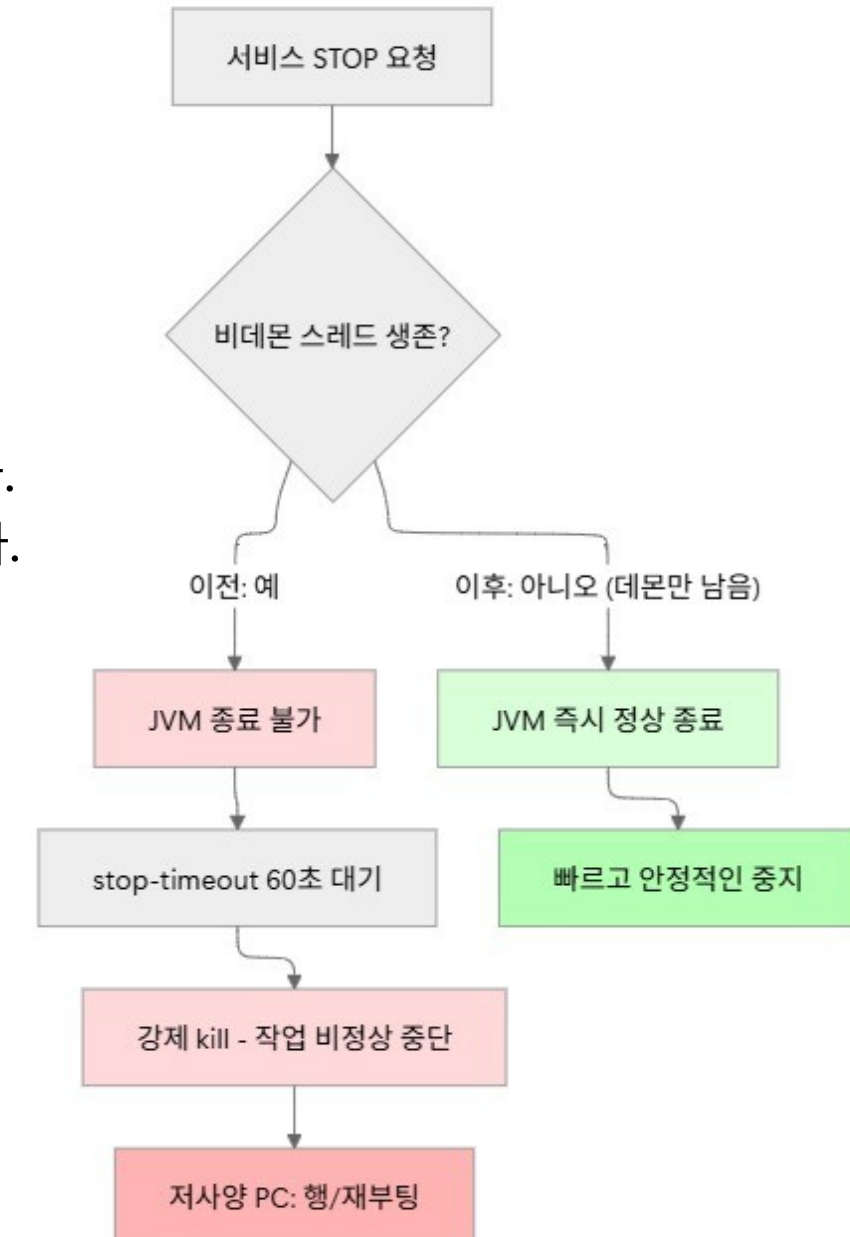
원인 분석

JVM 종료의 기본 규칙

JVM은 살아있는 비데몬(non-daemon, user) 스레드가 하나라도 남아 있으면 프로세스를 종료하지 않는다. main이 끝나고 Spring Context가 닫혀도, 사용자 스레드가 무한 루프를 돌고 있으면 JVM은 계속 살아있다.

JVM 종료 조건 = (모든 non-daemon 스레드 종료) OR System.exit()/강제 kill

데몬 스레드는 이 계산에서 제외된다. 즉 "남은 게 데몬 스레드뿐"이면 JVM은 즉시 종료한다.



원인 분석

왜 "어느 순간부터" 느려졌을까? 나름대로 추측을 해봤을때..

초기 버전에는 이런 상주 스레드가 적었지만, 기능이 늘면서 상주 백그라운드 스레드와 스케줄러가 누적된걸로 추측된다.

특히 최근에 우리팀 로드맵에 LLM/문서생성 기능이 들어오면서:

- @EnableScheduling / @EnableAsync (AcstApplication.java)
- LLMScheduler, LLMWebClientScheduler (@Scheduled 다수, 1~2초 주기 폴링)
- WebClient(Reactor Netty) 이벤트 루프
- SSE Emitter (SseEmitterManager)
- 곳곳의 new Thread(...).start() (DocumentTemplateGenerator 7곳, TraceService 3곳 등)



비동기·백그라운드 작업이 증가

즉, 기능 확장으로 서버 내부 실행 구조가 복잡해지면서 종료 관리 대상이 증가한 상황.

스케줄러/리액터 스레드는 대부분 Context 종료 시 정리되거나 데몬 스레드라 직접적인 원인은 아님.

하지만 스레드 수가 늘수록 종료 시퀀스가 무거워지고,
비데몬 스레드 때문에 JVM이 자발적으로 못 내려가는 현상이 점점 두드러지게 된 것으로 보인다.

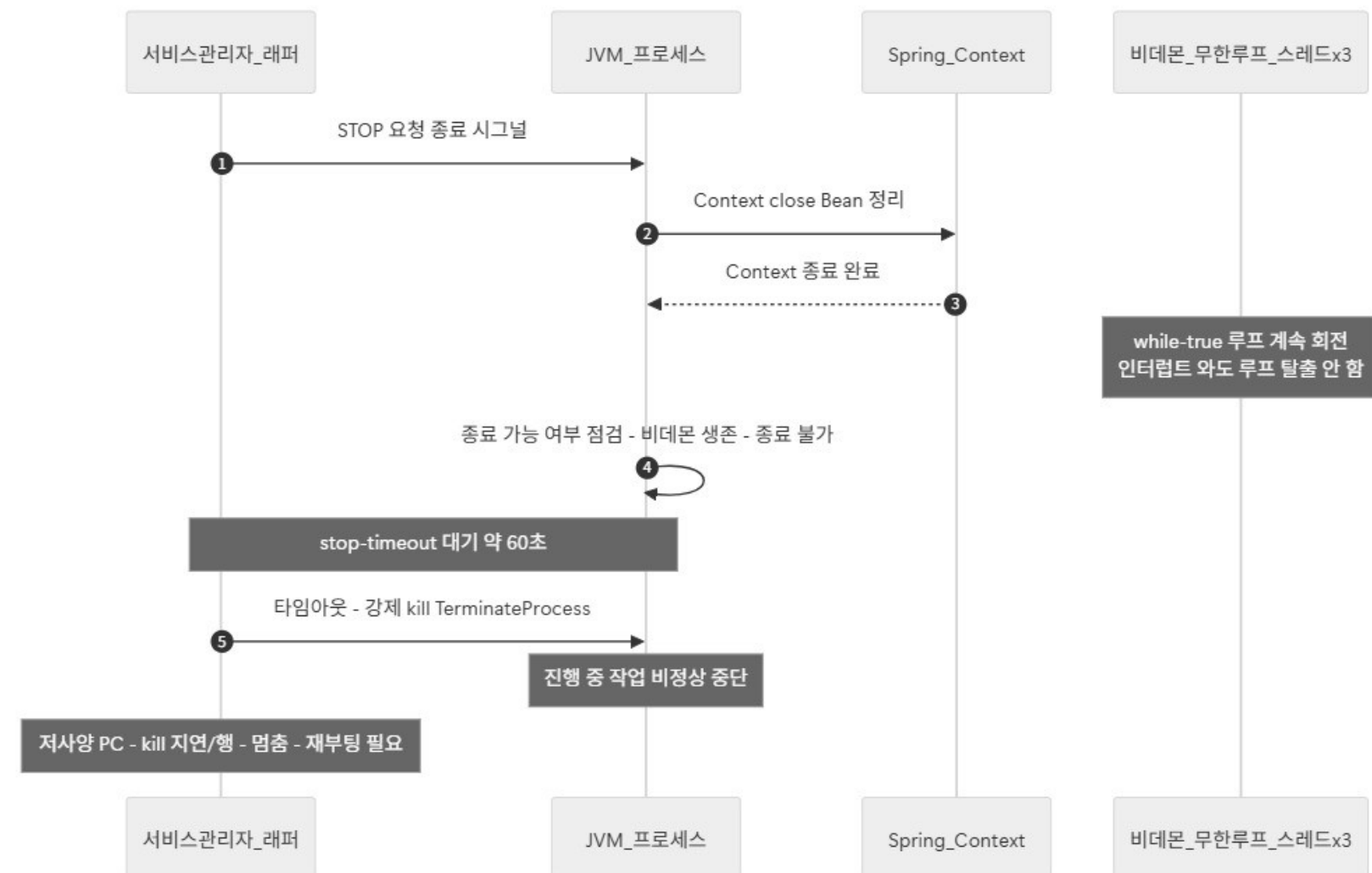
02 원인분석

원인 분석

서비스 종료 시퀀스 — 왜 "멈춤"으로 보일까?

서비스 관리자가 중지를 요청하면 다음 순서로 흐른다.
자발적 종료가 안 되므로
서비스 래퍼의 **stop-timeout**(현장 기준 약 60초) 이 만료된 뒤에야
강제 종료(TerminateProcess/kill) 가 일어난다.

다만 저사양 PC에서는
이 강제 종료 처리마저 지연되면서
서비스가 행(hang) 상태로 보이게 되는것으로 추측된다.



선택지 비교

원인 분석 결과,
서비스 종료 지연은 Tomcat/Catalina 종료 자체의 문제가 아니라 일부 장수 스레드가 JVM 종료를 막는 구조에서 발생한 것으로 확인된다.
이에 따라 해결 방안은 크게 두 가지로 생각해보았다.

방안	내용	장점	단점
A. 근본 리팩토링	종료 플래그 + interrupt() 기반 graceful shutdown, SmartLifecycle/@PreDestroy로 정리	정석. 진행 중 작업을 안전하게 마무리	3개 스레드 + 자식 워커 + 스케줄러까지 종료 연동 필요, 회귀 리스크 큼, 테스트 부담
B. 데몬 스레드 전환	각 스레드에 setDaemon(true) 1줄 추가	변경 최소(4줄), 회귀 위험 거의 없음, 즉시 효과	종료 시 진행 중 작업은 마무리 없이 중단됨

선택지 비교

B(데몬화)가 합리적이라고 판단했다.

근본 원인인 "누수(종료 안 되는 상주 스레드)"를 제대로 고치려면 스레드 생명주기 전체를 재설계해야 하는데 이는 영향 범위가 너무 크다고 생각했다.

반면 이 3개 문제 스레드가 하는 일은 모두 비핵심 + 멍등 + 재기동 시 복구 가능한 성격이다:

- 가비지 파일 청소: 다음 부팅 때 다시 스캔하면 됨
- 세션 동기화: 주기 폴링이라 한 사이클 누락돼도 다음에 보정
- 업로드 잡 큐 폴링: 처리 안 된 잡은 큐/DB 상태로 남아 재처리 가능

따라서 "종료 시점에 칼같이 끊어도 데이터 정합성이 깨지지 않는다"는 판단 하에,
로그로 식별된 누수 스레드들을 데몬 처리하는 것이 비용 대비 가장 안정적인 선택이라고 판단했다.

03 해결 방안 결정

실제 변경 (커밋)

결정한 B(데몬화)안 적용

JVM 종료를 막고 있던 3개 장수 스레드를 데몬 스레드로 전환하였다.

각 스레드의 start() 호출 직전에 setDaemon(true)를 추가하여, 서비스 종료 시 해당 스레드가 JVM 종료를 막지 않도록 수정했다.

```
public static AcstProjectGarbageFileThreadProcessor getInstance() {
    if (processorInstance == null) {
        processorInstance = new AcstProjectGarbageFileThreadProcessor();
        processorInstance.setDaemon(true); // 서비스 종료 시 JVM 종료를 막지 않도록 데몬 스레드로 기동
        processorInstance.start(); // Thread Start
    }
    return processorInstance;
}
```

AcstProjectGarbageFileThreadProcessor

프로젝트 삭제 시 남는 가비지 폴더/파일을
주기적으로(0.5초) 청소하는 싱글톤 스레드

```
15 15 @Component
16 16 public class SessionManagement extends Thread {
17 17
18 18     private static Map<String, HttpSession> sessionList = new LinkedHashMap<String, HttpSession>();
19 19     private static SessionManagement instance = new SessionManagement();
20 20
21 21     @Autowired
22 22     private SessionService sessionService;
23 23
24 24     @PostConstruct
25 25     public void managementRun() {
26 26         this.setDaemon(true); // 서비스 종료 시 JVM 종료를 막지 않도록 데몬 스레드로 기동
27 27         this.start();
28 28     }
}
```

SessionManagement

3초마다 전체 세션을 백엔드와
비교 동기화하는 싱글톤 스레드

```
public static ThreadWorkerProcessor getInstance() {
    if (instance == null) {
        instance = new ThreadWorkerProcessor();
        instance.setDaemon(true); // 서비스 종료 시 JVM 종료를 막지 않도록 데몬 스레드로 기동
        instance.start();
    }
    return instance;
}
```

ThreadWorkerProcessor

업로드 잡 큐를 0.1초마다 폴링하여
처리하는 싱글톤 스레드

04 적용 결과

SURESOFT

적용 결과

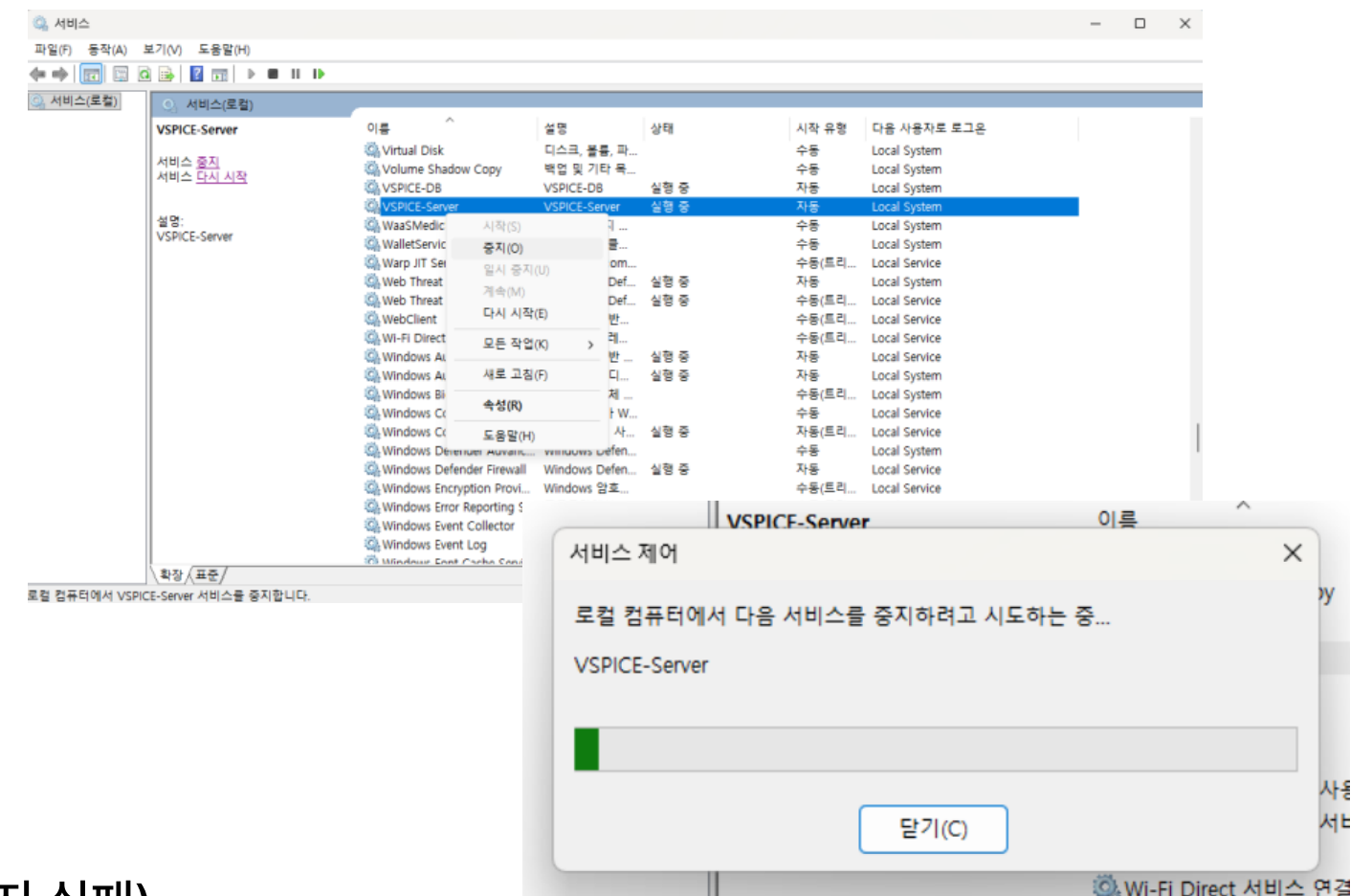
STOP signalled → Run service finished 까지의 실측

약 3분 정도 걸리던 서비스 종료 시간에서 확실히 개선되었다.

종료 시각	STOP 발행	Run service finished	소요	60초 timeout
2026-06-15 13:17	13:17:46	13:17:49	3초	미도달 OK
2026-06-15 13:29	13:29:00	13:29:04	4초	미도달 OK
2026-06-15 14:45	14:45:46	14:45:51	5초	미도달 OK
2026-06-15 17:00	17:00:21	17:00:27	6초	미도달 OK
2026-06-18 08:24	08:24:04	08:24:11	7초	미도달 OK

→ 패치 이전: 231~372초 (StopTimeout 60초 만료 후 강제 kill, 측정에 따라 중지 실패)

→ 패치 이후: 전 종료 **3~7초** 자연 종료. 단발 아닌 5회 재현.



04 적용 결과

검증 항목 수립

검증 방법

하지만 스레드를 건드는 작업이기 때문에 단순 OK가 아닌,
패치 이후 프로젝트가 전처럼 잘 작동하는지 꼼꼼한 검사가 필요했다.

#	검증 항목
①	기동 후 정상 동작
②	STOP 후 60초 타임아웃 없이 수 초 내 종료
③	상주 스레드 로그가 60초간 이어지지 않음
④	비데몬 스레드가 JVM 종료를 막지 않음



그래서 마련한 검증 항목들이다.

04 적용 결과

SURESOFT

검증 결과

① 기동 후 정상 동작

정상 출력 후, 동일 세션 안에서 ~2시간 운영하다 정상 종료.
기동~종료가 한 파일에 연속 기록되어 비정상 재기동·예외 없음.



```
[2026-06-15 14:46:18] [info] [ 4612] Apache Commons Daemon procrun (1.5.0.0 64-bit) started.  
[2026-06-15 14:46:18] [info] [ 4612] redirectStdStreams() stdout and stderr reassigned  
[2026-06-15 14:46:18] [info] [ 4612] Running Service 'VSPICE-Server'...  
[2026-06-15 14:46:18] [info] [ 5800] Starting service...  
[2026-06-15 14:46:19] [info] [ 5800] Service started in 1378 milliseconds. ← 기동 1.4초, 에러 없음  
[2026-06-15 17:00:21] [info] [28324] Console SHUTDOWN event signaled. ← ~2h14m 정상 운영 후 종료 (동일 세션)  
[2026-06-15 17:00:21] [info] [28324] Stopping service...  
[2026-06-15 17:00:21] [info] [28324] Stopping service...timeout 60000  
[2026-06-15 17:00:21] [info] [28324] Stopping service...timeout 60000 _jni_shutdown  
[2026-06-15 17:00:27] [info] [28324] Service stop thread completed.  
[2026-06-15 17:00:27] [info] [ 4612] Run service finished. ← 종료 6초, 단일 세션 정상 마감  
[2026-06-15 17:00:27] [info] [ 4612] Apache Commons Daemon procrun finished.
```

(commons-daemon.2026-06-15.log 기준)

04 적용 결과

SURESOFT

검증 결과

② STOP 후 60초 타임아웃 없이 수 초 내 종료

STOP(08:24:04) → finished(08:24:11) = 7초.

5번 정도 STOP Test를 해봤을때 3~7초, 모두 60초 미만으로 단발이 아닌 반복 재현.

패치 이전 231~372초 대비 명확히 개선.



```
[2026-06-18 08:24:04] [info] [ 5068] Service SERVICE_CONTROL_STOP signalled. ← STOP 발행 (기준점)
[2026-06-18 08:24:04] [info] [ 5124] Stopping service...
[2026-06-18 08:24:04] [info] [ 5124] Stopping service...timeout 60000
[2026-06-18 08:24:04] [info] [ 5124] Stopping service...timeout 60000 _jni_shutdown
[2026-06-18 08:24:10] [info] [ 5124] Service stop thread completed. ← stop 스레드 완료(~6초)
[2026-06-18 08:24:11] [info] [ 5068] Run service finished. ← 완전 종료(STOP +7초)
[2026-06-18 08:24:11] [info] [ 5068] Apache Commons Daemon procrun finished. ← procrun 정상 종료
```

(commons-daemon.2026-06-16.log 기준)

04 적용 결과

SURESOFT

검증 결과

③ 상주 스레드 로그가 60초간 이어지지 않음

마지막 스레드 경고(09.933) → destroy(10.024) → JVM exit(11)까지 약 1초.
경고가 60초 동안 반복되지 않고 destroy 직후 종료.



```
18-Jun-2026 08:24:04.908 INFO [Thread-36] AbstractProtocol.pause ["http-nio-38080"] 일시 정지 중 ← 종료 시작
18-Jun-2026 08:24:04.923 INFO [Thread-36] StandardService.stopInternal 서비스 [Catalina] 중지
18-Jun-2026 08:24:05.228 WARNING [Thread-36] clearReferencesThreads [RtmsLinkServer] [Thread-2] 스레드 중지 실패
com.suresoft.qscroll.acst.link.server.processor.ThreadWorkerProcessor.run(ThreadWorkerProcessor.java:111)
18-Jun-2026 08:24:09.932 WARNING [Thread-36] clearReferencesThreads [vspice] [Thread-33] 스레드 중지 실패
com.suresoft.qscroll.acst.util.Session.thread.SessionManagement.run(SessionManagement.java:75)
18-Jun-2026 08:24:09.932 WARNING [Thread-36] clearReferencesThreads [vspice] [Thread-34] 스레드 중지 실패
com.suresoft.qscroll.acst.util.AcstProjectGarbageFileThreadProcessor.run(AcstProjectGarbageFileThreadProcessor.java:100)
18-Jun-2026 08:24:09.933 SEVERE [Thread-36] checkThreadLocalMapForLeaks [vspice] netty InternalThreadLocalMap (무해, 재
내) ... ×2
18-Jun-2026 08:24:09.951 INFO [Thread-36] AbstractProtocol.stop ["http-nio-38080"] 중지 ← 마지막 경고 직후 stop
18-Jun-2026 08:24:10.024 INFO [Thread-36] AbstractProtocol.destroy ["http-nio-38080"] 소멸 | ← Tomcat 종료 완료
(이후 Thread-36 의 추가 스레드 경고 없음 → commons-daemon 08:24:11 JVM exit)
```

(catalina.2026-06-18.log 기준)

04 적용 결과

SURESOFT

검증 결과

④ 비데몬 스레드가 JVM 종료를 막지 않음

timeout 60000 이 설정되었음에도 60초 전(7초)에 Run service finished 발생 → 강제 kill 경로 미진입.
clearReferencesThreads 경고로 스레드 생존이 표시되더라도 JVM exit 를 막지 못했음이 교차 입증됨.

PASS

```
[2026-06-18 08:24:04] [info] [ 5124] Stopping service...  
[2026-06-18 08:24:04] [info] [ 5124] Stopping service...timeout 60000  
[2026-06-18 08:24:04] [info] [ 5124] Stopping service...timeout 60000 _jni_shutdown  
[2026-06-18 08:24:10] [info] [ 5124] Service stop thread completed.  
[2026-06-18 08:24:11] [info] [ 5068] Run service finished.
```

(commons-daemon.2026-06-18.log 기준)

```
18-Jun-2026 08:24:05.228 WARNING clearReferencesThreads [RtmsLinkServer] [Thread-2] 쓰레드 중지 실패  
18-Jun-2026 08:24:09.932 WARNING clearReferencesThreads [vspice] [Thread-33] 쓰레드 중지 실패  
18-Jun-2026 08:24:09.932 WARNING clearReferencesThreads [vspice] [Thread-34] 쓰레드 중지 실패
```

(catalina.2026-06-18.log 기준)

← ~6초 만에 stop 완료
← StopTimeout(60s) 미도달, 자연 종료

로그 대조

같은 시각 catalina는 아직 스레드가 살아있다고 경고하지만,
commons-daemon은 그럼에도 60초 강제 kill 없이 종료

↑ 그럼에도 StopTimeout 미도달, 08:24:11 종료

안정성 논거

1. 데몬 전환은 "런타임 동작"을 바꾸지 않는다

데몬 스레드와 사용자 스레드는 실행 중에는 완전히 동일하게 동작한다.

우선순위, 스케줄링, 동기화, 작업 내용 무엇도 달라지지 않는다. 유일한 차이는 JVM 종료 시점의 정책뿐이다:

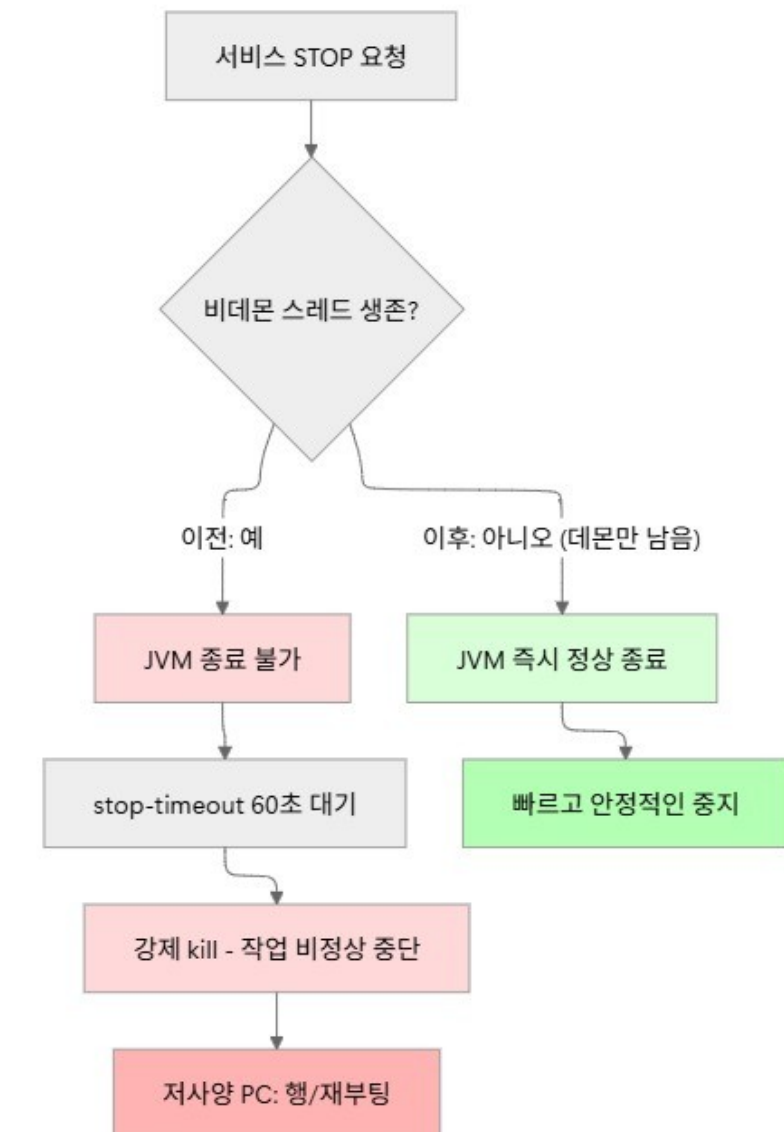
- (이전) 비데몬: JVM이 이 스레드가 끝나길 기다림 → 영원히 안 끝남 → 타임아웃 → 강제 kill
- (이후) 데몬: JVM이 이 스레드를 기다리지 않음 → 다른 비데몬이 없으면 즉시 정상 종료

2. "강제 kill 되던 작업"을 "자발적 종료"로 바꾼 것

기존에도 이 스레드들은 종료 시 graceful하게 끝나지 못하고 결국 60초 뒤 강제 kill로 죽었다.

즉 진행 중 작업이 중단되는 것은 변경 전후가 동일하다.

데몬화는 그 강제 kill을 JVM의 즉시 자발적 종료로 앞당긴 것일 뿐이므로, 안정성을 떨어뜨리지 않으면서 종료 지연만 제거한다.



04 적용 결과

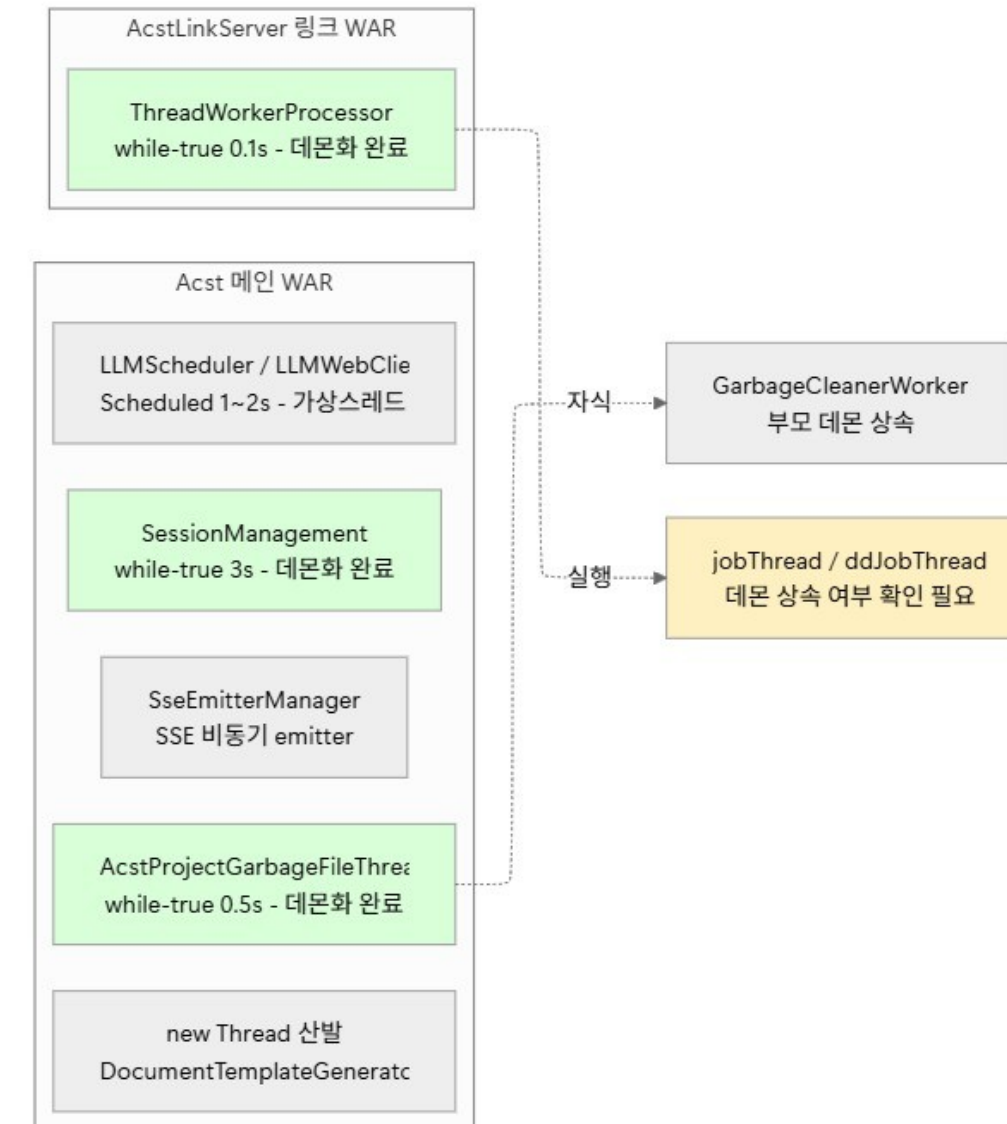
부수 효과

자식 스레드 데몬 상속

Java에서 새 스레드는 자신을 생성한 스레드의 데몬 속성을 상속한다.

AcstProjectGarbageFileThreadProcesser.run() 내부에서
new Thread(new GarbageCleanerWorker(...))로 생성되는 워커들은,
이제 부모가 데몬이므로 자동으로 데몬으로 생성된다.

따라서 별도 수정 없이 가비지 청소 워커들까지 JVM 종료를 막지 않게 되며,
서비스 종료 안정성이 함께 개선되는 추가 효과가 있다.



04 적용 결과

SURESOFT

결론

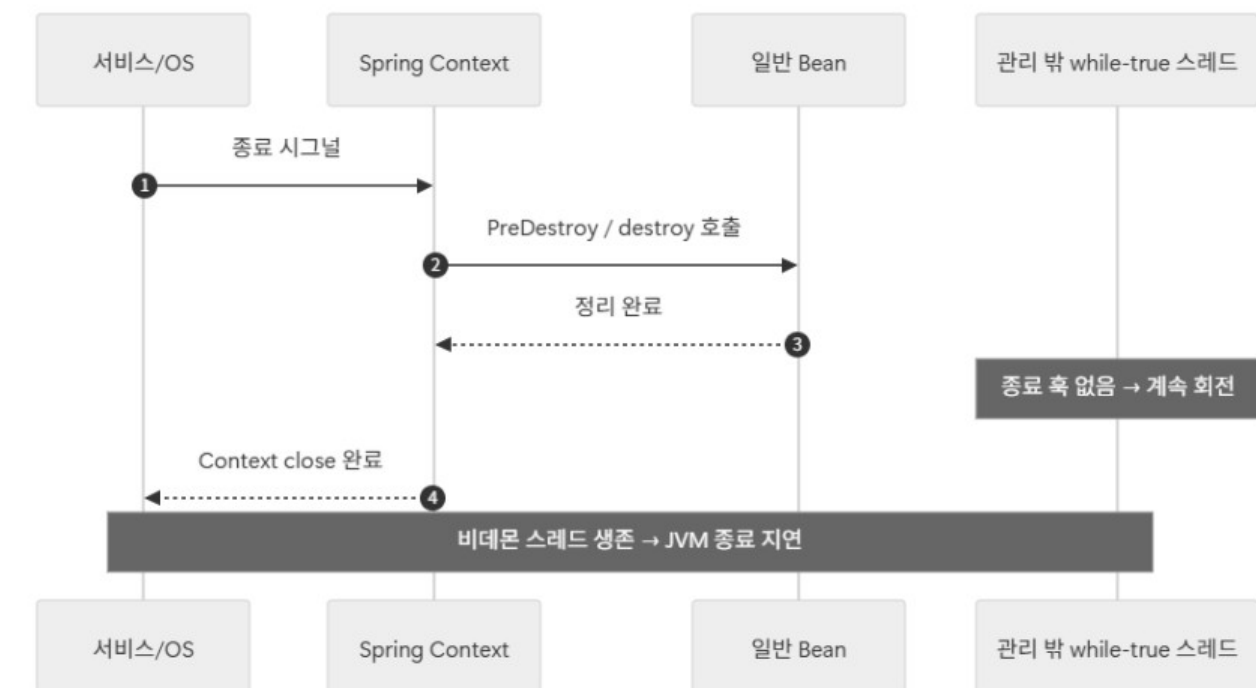
결론 정리 및 후속 조치 필요성

종료 지연·멈춤의 진짜 원인은 "종료 조건이 없는 비데몬 상주 스레드"라는 누수였다.

누수 자체를 근본 제거하는 것은 큰 작업이라, 로그로 식별한 핵심 3개 스레드를 데몬 전환하여 JVM이 강제 kill 없이 즉시 정상 종료되도록 했다.

런타임 동작은 그대로 두고 종료 정책만 바꾼,
영향 범위 최소·회귀 위험 최소의 안정적 처방이다.

이는 영향 범위와 회귀 위험을 최소화한 임시적인 조치이며,
명시적인 종료 조건을 갖춘 graceful shutdown 구조화는 후속 과제로 남은것으로 보인다.



감사합니다!